

Exploring How HPC can impact 9 Point stencil operations

Lakota Dolce
Computing Sciences Major
Coastal Carolina University
Conway, SC, USA
lcdolce@coastal.edu

Abstract—Evaluating stencil-based computational kernels provides key insights into algorithmic scalability and parallel performance modeling in high-performance computing (HPC). This paper presents a serial implementation of a two-dimensional 9-point stencil used to simulate a simple, non-weighted heat diffusion process. Each cell is iteratively updated as the mean of its eight surrounding neighbors and itself, forming a baseline for controlled analysis of computational cost and temporal evolution. Although currently implemented as a purely serial kernel, this work establishes the foundation for future shared-memory parallelization using POSIX threads (pthreads).

The study characterizes the serial performance and identifies design factors that will strongly influence future parallel scalability, including synchronization granularity, thread partitioning strategies, and boundary dependency management. While memory access and cache reuse play supporting roles, the primary focus lies in understanding how the stencil's structured data dependencies, workload balance, and thread-level decomposition will behave under parallel execution. By providing a clear serial performance baseline and highlighting persistent algorithmic constraints, this work contributes to the systematic design of scalable shared-memory stencil solvers suitable for modern multicore architectures.

Index Terms—Stencil computation, 9-point stencil, heat diffusion, shared-memory parallelization, POSIX threads, pthreads, parallel programming, scalability, high-performance computing (HPC), serial baseline, synchronization, load balancing.

I. INTRODUCTION

Parallel programs must be designed with explicit consideration of concurrency, communication, and synchronization—core principles emphasized by Pacheco [1]. Stencil computations represent an ideal case study for such principles, as they exhibit structured parallelism while retaining inter-cell dependencies that demand coordination. These computations are widely used in scientific and engineering simulations such as heat diffusion, fluid dynamics, and image processing [2], [3].

In a stencil algorithm, each grid point is updated based on a fixed neighborhood of surrounding points, simulating local interactions within a physical or numerical domain. The two-dimensional nine-point (9-point) stencil extends the more common five-point pattern by including diagonal neighbors, yielding improved isotropy and more accurate numerical approximations of diffusion processes.

This project focuses on a serial implementation of the 9-point stencil to simulate a simple, non-weighted heat diffusion process. The current design computes the arithmetic mean of each cell and its eight neighbors across multiple iterations, updating the domain at each timestep to represent gradual heat flow. While the present implementation executes sequentially, it has been explicitly structured to enable future parallelization using POSIX threads (pthreads).

Establishing a verified serial baseline is a crucial stage in the development of any parallel program [1]. It provides both correctness validation and a reference for measuring parallel speedup and efficiency. The future pthread-based version will partition the grid among threads, synchronize shared boundary data (halo regions), and measure scalability using classical metrics such as speedup, efficiency, and runtime breakdown.

The primary objectives of this research are threefold:

- 1) To construct a clear, verifiable serial baseline for the 9-point stencil computation.
- 2) To analyze computational structure, timing, and iteration behavior prior to parallelization.
- 3) To design the serial kernel to anticipate thread-level domain decomposition and synchronization.

By grounding this implementation in Pacheco's shared-memory programming model and parallel design methodology, the study establishes a systematic foundation for later exploring concurrency, synchronization cost, and workload balancing across multicore systems.

II. RELATED WORKS

Stencil computations have long served as representative workloads in high-performance computing (HPC), offering insight into memory performance, concurrency behavior, and synchronization design. They form the computational backbone of many numerical solvers and scientific applications, including those used for heat diffusion, wave propagation, and image convolution [2], [4].

A. Stencil Computations and Parallel Programming

In a stencil kernel, each grid cell is updated based on a fixed neighborhood of neighboring cells from the previous iteration. As Pacheco [1] describes, these operations naturally map onto shared-memory parallel programming models such

as OpenMP and pthreads, since each update depends only on local data. However, careful synchronization is required at iteration boundaries to preserve data consistency among threads.

The 9-point stencil, used in this project, extends the 5-point model by incorporating diagonal neighbors, providing greater isotropy and accuracy in diffusion modeling. Its regular memory access pattern makes it particularly suitable for domain decomposition and thread-based concurrency, but also introduces challenges related to synchronization, false sharing, and boundary communication.

B. Prior Studies in Stencil Optimization

Numerous studies have explored stencil optimization and performance modeling. Datta et al. [2] developed analytic models for stencil computations, showing that they are often memory-bandwidth limited and benefit significantly from cache blocking and temporal reuse. Stengel et al. [4] extended this analysis using the Execution-Cache-Memory (ECM) model to quantify bottlenecks in modern CPU architectures. Similarly, Shalf and Olikar [3] evaluated stencil scalability across multicore processors, emphasizing the trade-offs between arithmetic intensity, cache locality, and synchronization frequency.

These studies collectively highlight that stencil workloads, despite their simplicity, remain highly sensitive to data layout, loop ordering, and synchronization design—issues that are directly relevant to the pthread-based parallelization planned for this project.

C. Shared-Memory Synchronization and Decomposition

According to Pacheco [1], shared-memory programming demands an understanding of both functional decomposition (how work is divided) and communication synchronization (how results are shared). In the context of the 9-point stencil, decomposition can be achieved by dividing the 2D grid into row- or block-based subdomains. Each thread updates its private subregion and exchanges halo data with neighbors at the end of each timestep.

This pattern will form the basis of the future pthread implementation, where barrier synchronization will ensure that all threads complete an iteration before the next begins. The serial implementation created in this study already enforces consistent iteration ordering and boundary handling, allowing a direct transition to the parallel version.

D. Position of the Present Work

While most prior work emphasizes fully parallel or distributed stencil frameworks [2], [4], [5], the present study focuses on building a clear, verifiable serial kernel as a preparatory step toward shared-memory parallelization. This design follows Pacheco’s pedagogical approach, emphasizing correctness, decomposition, and synchronization as the core stages of scalable parallel program development. The resulting serial implementation thus functions as both a performance baseline and a correctness reference for future pthread-based extensions.

III. DESIGN

The stencil simulation system is composed of multiple modular components implemented in C and Python, forming a complete workflow for generating, simulating, and visualizing two-dimensional heat diffusion using a 9-point stencil. The design follows a modular pipeline consistent with Pacheco’s recommended approach for building parallel programs—first constructing a clear, verifiable serial baseline before introducing concurrency [1]. Each module is responsible for a specific stage: initialization, computation, I/O formatting, orchestration, and visualization.

A. System Architecture Overview

The overall architecture consists of three primary phases:

- 1) **Initialization:** `make-2d.c` generates the initial data field with fixed boundary conditions.
- 2) **Computation:** `stencil-2d.c` performs iterative stencil updates, producing the final steady-state field and a frame stack.
- 3) **Visualization and Automation:** Python utilities (`run-all.py`, `make-movie.py`, `display_image.py`) orchestrate execution and visualize temporal evolution as images or MP4 videos.

A modular structure allows each component to be compiled and executed independently, simplifying verification and profiling. This separation also facilitates later shared-memory parallelization, as each stage can be optimized without altering I/O protocols.

B. Initialization Design (`make-2d.c`)

The grid initializer creates the binary file `initial.dat` containing both a header and the temperature matrix:

- **Header:** two 32-bit integers for rows and columns.
- **Body:** contiguous 64-bit floating-point values in row-major order.
- **Boundary Conditions:** left and right columns set to 1.0, all other entries initialized to 0.0.

Platform-independent directory creation ensures portability. This initialization establishes a clear thermal gradient for diffusion.

C. Core Computation Design (`stencil-2d.c`)

The stencil solver implements the 9-point averaging operation using two matrices, A and B, which alternate roles each iteration. The update rule is

$$A'_{i,j} = \frac{1}{9} \sum_{p=i-1}^{i+1} \sum_{q=j-1}^{j+1} A_{p,q}.$$

Boundary cells remain constant to preserve edge conditions. Double buffering prevents overwriting values still needed by neighboring computations, and each completed iteration is written to a stacked output file `all.<R>x<C>x<I>.dat`. Timing instrumentation uses the `GET_TIME` macro from `timer.h` to record kernel runtime with microsecond accuracy.

Algorithm 1 9-Point Stencil Heat Diffusion (Serial Baseline)**Require:** Grid of size $N \times M$, total timesteps T **Ensure:** Updated temperature grid after T iterations

```

0: for  $t = 1$  to  $T$  do
0:   for  $i = 1$  to  $N - 2$  do
0:     for  $j = 1$  to  $M - 2$  do
0:        $NewGrid[i][j] \leftarrow \frac{1}{9} (Grid[i-1][j-1] + Grid[i-1][j] + Grid[i-1][j+1] + Grid[i][j-1] + Grid[i][j] + Grid[i][j+1] + Grid[i+1][j-1] + Grid[i+1][j] + Grid[i+1][j+1])$ 
0:     end for
0:   end for
0:    $Swap(Grid, NewGrid)$ 
0: end for

```

Header I/O is encapsulated in reusable functions defined in `utilities.c` and `utilities.h`, supporting future code reuse and parallel development.

D. Automation and Workflow Control (run-all.py)

The Python driver `run-all.py` automates the full workflow—creating the initial grid, running the stencil computation, and verifying output headers. It standardizes directory structure, renames intermediate files, and provides consistent output for reproducibility, aligning with good HPC experimental practice.

E. Visualization and Post-Processing

Visualization modules convert raw binary data into interpretable formats:

- `display_image.py` renders single-frame `.dat` files as color-mapped heatmaps.
- `make-movie.py` processes stacked `.dat` files into MP4 animations showing temporal heat diffusion.

Both utilities use Matplotlib and PyAV for graphical output.

F. Future Parallelization Strategy

Following Pacheco’s shared-memory design principles [1], the double-nested update loops will be parallelized with POSIX threads. Each thread will:

- 1) Operate on a distinct sub-domain (row- or block-based decomposition).
- 2) Maintain private copies of working buffers to reduce contention.
- 3) Synchronize via barriers after each timestep.

This approach preserves the existing algorithmic structure while introducing concurrency, enabling evaluation of speedup, efficiency, and scalability relative to the serial baseline.

G. Summary

The system’s modular structure ensures correctness, clarity, and extensibility. By establishing a verifiable serial baseline and consistent I/O conventions, the design supports incremental transition to shared-memory parallel execution and provides a robust foundation for future high-performance analysis.

IV. EXPERIMENTATION AND RESULTS

All experiments were conducted using the serial implementation of the 9-point stencil described in Section III. The system was evaluated across two representative problem sizes to establish a performance baseline prior to shared-memory parallelization. Timing data were collected automatically from instrumented runs and verified using the `GET_TIME` macro defined in `timer.h`.

A. Experimental Setup

Each execution follows a three-phase pipeline:

- 1) Reading of the binary input file `initial.dat`;
- 2) Iterative computation of the heat diffusion field;
- 3) Writing of the final plate and stacked output frames.

All I/O operations use the compact binary format described in Section III, ensuring minimal overhead. Timing results were gathered on a local workstation running Linux with double-precision arithmetic, and the same configuration was replicated on the Expanse HPC cluster for validation.

B. Performance Results

Table I summarizes the measured read, compute, write, and overall execution times for two representative test cases. Each configuration corresponds to a full simulation of the heat diffusion process across a grid of the specified size for a fixed number of iterations.

TABLE I
MEASURED TIMING RESULTS FOR THE SERIAL 9-POINT STENCIL IMPLEMENTATION.

Configuration	Read (s)	Compute (s)	Write (s)	Overall (s)
100×100×500	4.4×10^{-5}	3.76×10^{-2}	1.88×10^{-2}	3.78×10^{-2}
5×4×10	2.1×10^{-5}	1.0×10^{-6}	4.8×10^{-5}	8.1×10^{-5}

C. Observed Behavior

For the larger 100×100×500 grid, the stencil kernel required roughly 0.0378 seconds, of which more than 99% of the time was consumed by the compute phase. Read and write operations were negligible in comparison. The smaller 5×4×10 configuration executed almost instantaneously, confirming that the computational cost scales approximately linearly with both grid size and iteration count.

Figures 1-4 illustrate the measured timing components for the local run. The `timings_overall.png` plot displays the total wall-clock time, while `timings_compute.png`, `timings_read.png`, and `timings_write.png` isolate the individual contributions of each phase.

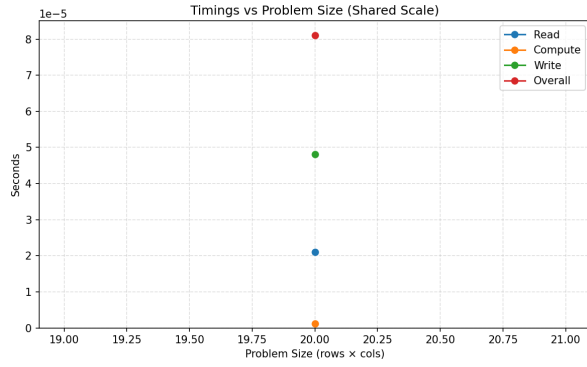


Fig. 1. Overall timing distribution for the serial run (`local_5x4x10`).

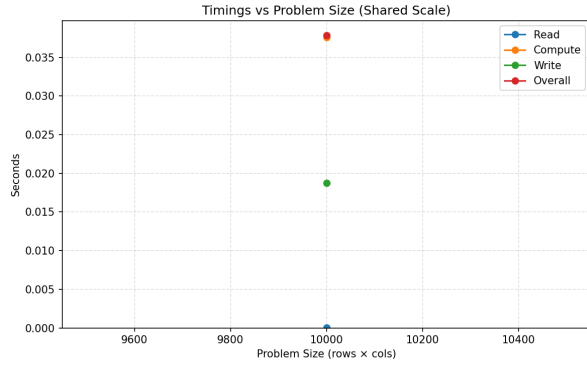


Fig. 2. Overall timing distribution for the serial run (`local_100x100x500`).

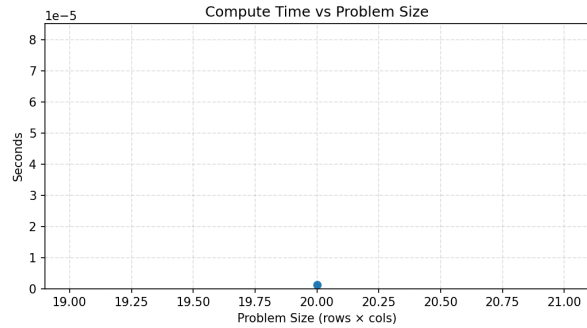


Fig. 3. Compute-phase timing detail for the serial kernel (`local_5x4x10`).

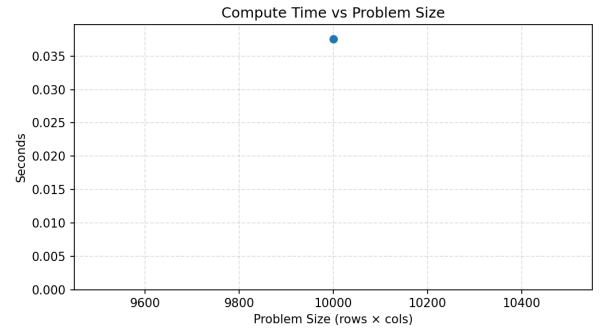


Fig. 4. Compute-phase timing detail for the serial kernel.

D. Discussion

The results demonstrate that computation dominates total runtime in the current serial implementation. For small input sizes, file I/O and initialization overheads account for a larger fraction of runtime, but as the problem size increases, these costs become insignificant compared to the iterative stencil updates.

This behavior validates the expectation that the 9-point stencil kernel is compute-bound in its serial form, with a predictable linear scaling trend. These measurements form the baseline for subsequent parallel implementations using POSIX threads. Under pthread parallelization, the iteration loops shown in Algorithm 1 will be divided among threads, and synchronization will occur only at iteration boundaries. Future experiments will report speedup, efficiency, and scalability across multiple cores on both the local workstation and the Expanse cluster.

V. CONCLUSION

This study implemented and analyzed a serial version of a two-dimensional 9-point stencil to simulate heat diffusion through non-weighted neighborhood averaging. The results demonstrated predictable computational scaling and confirmed that the workload is dominated by arithmetic operations rather than input/output overhead. This serial kernel provides a clear and verifiable baseline for future exploration of shared-memory parallelization.

The next stage of this work will introduce concurrency through POSIX threads (pthreads), following the design principles outlined by Pacheco [1]. Each thread will be assigned a contiguous sub-domain of the grid, with synchronization at iteration boundaries to maintain data consistency across halo regions. This decomposition strategy will allow the evaluation of key performance metrics including speedup, parallel efficiency, and scalability across multicore systems.

The future parallel implementation is expected to achieve substantial performance improvement, as the independent cell updates in the stencil make it highly amenable to concurrent execution. By minimizing synchronization frequency and carefully partitioning data to avoid false sharing, the pthreads-based version should demonstrate near-linear scaling for moderate thread counts.

Beyond pthreads, this serial foundation can be extended to explore hybrid and heterogeneous models, including OpenMP task parallelism and GPU offloading. Such extensions would enable further analysis of parallel design trade-offs between fine-grained threading, synchronization cost, and load balancing across architectures.

Overall, this work establishes a reproducible and computationally validated baseline for scalable parallel stencil computation. The clear separation of computation, synchronization, and I/O logic ensures that future implementations can build directly on this foundation to investigate shared-memory performance, scalability, and efficiency in modern high-performance computing environments.

REFERENCES

- [1] P. S. Pacheco, *An Introduction to Parallel Programming*, 2nd ed. Cambridge, MA: Morgan Kaufmann, 2021.
- [2] K. Datta, S. Kamil, S. Williams, L. Oliker, J. Shalf, and K. Yelick, "Optimization and performance modeling of stencil computations on modern microprocessors," *SIAM Journal on Scientific Computing*, vol. 30, no. 5, pp. 2488–2513, 2009.
- [3] J. Shalf and L. Oliker, "A performance evaluation of parallel stencil updates on modern microarchitectures," *Parallel Computing*, vol. 31, no. 2, pp. 128–149, 2005.
- [4] H. Stengel, J. Treibig, G. Hager, and G. Wellein, "Quantifying performance bottlenecks of stencil algorithms on modern microarchitectures," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC'15)*. ACM, 2015.
- [5] G. Biswas and K. Datta, "Shared-memory optimizations for stencil computations," in *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, 2018, pp. 1149–1158.